

MRR GCP marker detection.

By: Sean S.C.A. Winklaar, and Stijn S.C.A. de Graauw

Table of content

Table of content	1
Abstract	2
Research questions	2
Methods	3
How can GCP marker detection be integrated into the existing software of MRR in a robust, sustainable and scalable way?	4
What is the ideal GCP marker for detection?	4
How can GCP marker detection be done in a robust way?	6
How does the data need to be prepared for GCP marker?	15
How can the data best be used to detect future GCP marker?	22
Conclusion.....	28
Future Steps	28
Bibliography	29
Figures	30

Abstract

MRR drones is a company that specializes in photogrammetry for construction projects. The use for that photogrammetry is to do volume measurements of pits or mounds, to see how much fill is needed or how much stuff is in a mound.

The ground control point markers then, are to increase the accuracy of those photogrammetry models. Those markers are normally hand marked in the software which takes a lot of time. Thus, MRR drones wants this process to get automated thus leading to this research.

This led to a choice in marker type, a pipeline design, a data preparation method, and proof of concepts for 2 of the proposed models.

With these results, an advice was given on how to proceed with development of this automation project.

Research questions

Main Research Question

How can GCP marker detection be integrated into the existing software of MRR in a robust, sustainable and scalable way?

Sub-Questions

1. What is the ideal GCP marker for detection?
2. How can GCP marker detection be done in a robust way?
3. How does the data need to be prepared for GCP marker?
4. How can the data best be used to detect future GCP marker?

Methods

Question	method	Deliverable
RQ1	• Literature study • Available product analysis	• GCP marker candidates
RQ2	• Stakeholder analysis • User requirements • Expert interview • decomposition • Literature study • Available product (technology) analysis • Architecture sketching	• User requirements • List of promising technologies and methods • Proposed pipeline architecture
RQ3	• Literature study • Available product (technology) analysis • Data quality check • Prototyping	• Data preparation pipeline • Data quality assessment • Prepared data
RQ4	• Literature study • Good and bad practices • Prototyping • Model evaluation • Model validation	• Evaluated detection model

How can GCP marker detection be integrated into the existing software of MRR in a robust, sustainable and scalable way?

As the question suggests, the goal of this research is to detect ground control points (GCP's) in an automated way for MRR drones. MRR drones is a company that specializes in photogrammetry for construction projects. The use for that photogrammetry is to do volume measurements of pits or mounds, to see how much fill is needed or how much stuff is in a mound.

The GCP markers then, are to increase the accuracy of those photogrammetry models. Those markers are normally hand marked in the software which takes a lot of time. Thus, MRR drones wants this process to get automated thus leading to this research.

What is the ideal GCP marker for detection?

The goal of this section is to look at different GCP types to see if one has a clear edge in automated detection in the use case of MRR drones. To answer this question, an analysis of similar projects was done that looked at different markers to then compare the results. In this analysis any costs and other benefits or issues were also taken into account.

Literature study

This literature study focused on finding out what kind of GCP markers did well in other detection automation projects. Specifically, the shapes and colours of the markers.

(Chuanxiang, et al., 2023) found that, in the case of their white and red markers the cross performed better than an L shaped marker.

Harlequin and Checkerboard Targets

Evaluation of black and white quadrant GCP markers by (Odriozola, Pauly, Oswald, & Raymaekers, 2024) shows that machine learning detection improves significantly when models are trained with multiple tile sizes (224×224 and 512×512 pixels) rather than a single resolution. Using ResNet-152, these markers achieved sub-pixel accuracy with a mean absolute error of 0.586 pixels, proving robust against occlusion, glare, debris, and variable lighting.

Commented [SG1]: Image? Any licensing? What is pricing? Was there a difference in the mean absolute error between the 2? What is the identification rate like? What is the recall like? Are they available in multiple colours?

Commented [SG2R1]: Ask yourself these questions for all of the marker types

Cross-Type Markers

(Chuanxiang, et al., 2023) cross-shaped GCP markers achieved a mean average precision of 0.982, the highest among compared shapes. High-contrast geometric patterns are ideal for automated detection.

Commented [SG3]: Image?

Commented [SG4]: It's a hint in that direction yes. But it doesn't confirm it yet

SNDRT and PXCCT (coded targets) (Wang, et al., 2022)

While coded targets such as SNDRT and PXCCT allow automatic marker identification, their recognition depends on resolving fine geometric detail within the marker. At higher altitudes and lower image resolutions, fewer pixels represent each marker in UAV imagery, which reduces the reliability of decoding these structures. In contrast, simpler high-contrast designs such as cross or black-and-white quadrant targets remain more robust under these conditions, as their detection relies primarily on overall shape and contrast rather than fine internal patterns.

Available product analysis

UAV Harlequin Ground Control Points

\$99.99

(Sky high bulls eye, n.d.)

Checkerboard GCP markers

\$86.00

(amazon, n.d.)

Aerotas Ground Control Points (GCPs)

Different colour GCPs (checkerboard)

\$90.00

(Aerotas, n.d.)

Conclusion

According to the sources found the identification accuracy and marking accuracy of GCP markers of many types is relatively accurate. No marker type seems to have an outstanding accuracy. MRR currently uses numbered harlequin targets. Thus, to minimize costs those will be used for this project.

How can GCP marker detection be done in a robust way?

Automating GCP marker detection can save a lot of manual labour, but making it work reliably in the real world is difficult. Drone images are taken at different heights, in changing weather, and over messy construction sites. If the software misses a marker or calculates its centre incorrectly, the final 3D model will be inaccurate.

Therefore, the system must be robust enough to handle these challenges. This section looks at what the users need, what experts recommend, and how to design a software pipeline that detects markers accurately without making costly mistakes.

Stakeholder Analysis

What stakeholders are involved in this project will be important to know. Mainly for knowing in what context the detection will have to work, and what robustness means in those contexts. Thus, the stakeholders are as follows.

- **MRR drones:** the provider of the software infrastructure and drones that will be used for the detection
- **Construction companies:** the end users that will be putting down the GCP's and flying the drones
- **Civil developers:** the ones financially involved with the processes of construction companies.

Their interests are relatively wide ranging. construction companies and Civil developers have an interest in getting the detection as precise as possible. But this is only true up until it becomes more expensive to use the product than the material they save by using the product.

MRR drones on the other hand, needs it to be effective enough to outperform competitors while being as efficient as possible to host cheaply. This isn't in direct conflict with the other stakeholders but does bring a certain balancing act.

On top of this MRR wants the product to work as reliably as possible while costing less than a human to minimize labour costs.

Expert interview

There were 2 expert interviews done related to robust and ethical integration of ICT systems and where relevant computer vision specifically. The people who were interviewed are:

- **Rens M.C.M. van der Vorst:** A teacher at Fontys University with experience in analysing the societal impact of ICT products.
- **Coen C.J.H. Crombach:** A teaching at Fontys focusing on projects with LLM's, such as vibe coding and the "Virtual human" project.

Rens M.C.M. van der Vorst

When asked about important factors to pay attention to Rens listed the following:

- Goals of the project
- Privacy of users
- Benefits weighed against costs and risks
- Ethicality of current performance
 - o Accuracy
 - o Impact of a mistake
- Uses for bad actors

With this list of factors, it's clear that it is important that a product has a clear problem statement. That would already start a project with a clear goal in mind and a clear starting point for the benefits. This then needs to be weight against all kinds of costs, such as Energy usage (and the pollution from the generation thereof), impact on the lives of users, the risks around saving personal data, and the impacts of a mistake by the system.

The impact on the lives of users also includes the impact of bad actors using the service. A good example of this is stalking through social media. In that case the goal of keeping up with other people's lives, also allows the tracking of someone's movement based on their posts.

After the factors Rens was asked about where most deployed IT systems fail. He said that most IT systems fail because they are "the right solution for the wrong problem". By this he means that most products that fail missed an important aspect of the problem they were solving or overestimated how much of a problem it was. The first case leads to a product that's unusable, the second to a product with too little users to break even.

Rens was asked if there were any tools that help avoid these problems and help pay attention to all these factors. He recommended the TICT tool developed with the help of Fontys. He noted that Category 1 of the TICT is the most important as it pertains to the

problem statement. The TICT tool is a list of questions developed to help developers think about the problem they are solving and the societal impact of their solution.

Coen C.J.H. Crombach

The factors Coen listed as particularly important when making IT systems an integrating AI are being transparent about shortcomings of the components, and the impact on users. On the impact on users, he said the aim should be to increase efficiency in parts of a process that people don't like. An example of that is lessening the time spent on documenting in healthcare so healthcare workers have more time to spend with patients. That way your projects increase both worker efficiency and moral.

When it comes to points where deployed products fail most often Coen noted to have little direct experience with deployment. But he noted that it isn't uncommon for projects to have unforeseen costs as compared to their proof of concept. Another reason he has seen for projects failing is not being detailed enough when defining user requirements or not following requirements that were given by stakeholders.

For quality assurance and ethics tools that Coen would recommend, it comes down to any tool that helps start a valuable discussion on what the value and risks of the project are. For this the best situation would be a group of developers and stakeholders with a diverse set of worldviews. Specific tools to help with this that Coen named are the TICT tool again, and the moral design game. Another important thing to do with the risks found in these kinds of discussions is to see if these can be mitigated in any way.

User requirements

To clarify what the interests from all the stakeholders mean and how they should be balanced in our project, the requirements will be listed here in a more detailed and measurable format. They will be split up into functional and non-functional requirements and listed in order of priority with highest priority first.

FR1. The Precision of marking the GCP markers has to facilitate cm or more precise accuracy for the final photogrammetry model.

Q1. The marked spot has to be on the middle of the marker

FR2. The marking of the GCP markers should be reliable enough to save on labour costs.

Q1. On low certainty, human reassessment should be requested.

L1. TPR of marking of 0,9 or greater

L2. Recall of 0,8 or greater

L3. TNR of 0,9 or greater

L4. FPR of 0,1 or lower

Non-functional requirements

NFR1. Model should be compatible with tensor flow runners.

Decomposition

To develop a robust system for GCP marker detection, the problem can be decomposed into the following components of the detection pipeline:

- **Marker design (see RQ1)**
- **Data acquisition and variability**
- **Detection model**
- **Post-processing**
- **Evaluation and validation**
- **System integration and scalability**

Literature Study

Reliable detection of GCP markers in UAV imagery faces several key challenges. These include:

- Small object size, limiting visibility and detail at higher altitudes
- Environmental variability, such as changes in lighting, weather, and terrain
- Visually complex backgrounds that can obscure or resemble markers

Aerial object detection and photogrammetry provides insight into which methods and technologies address these challenges effectively. In this context, performance is defined based on the project's requirements, including high detection accuracy ($\text{TPR} \geq 0.9$, $\text{Recall} \geq 0.8$, $\text{TNR} \geq 0.9$, $\text{FPR} \leq 0.1$) and precise localization of the marker center.

Data Acquisition and Variability

The quality and variability of UAV imagery is strongly influenced by acquisition parameters, particularly flight altitude, lighting conditions, and weather. (Odriozola et al., 2024) explicitly identify drone flight altitudes, diverse lighting conditions, and potential obstructions on GCP markers as pivotal variables that must be represented in any training dataset used for GCP detection. They argue that the comprehensiveness and fidelity of the dataset in capturing these variables directly underpins the adaptability of the detection pipeline to real-world conditions.

Small Object Detection

A consistent challenge in UAV imagery is that GCP markers occupy very few pixels, especially at higher altitudes.

(Odriozola et al., 2024) showed that training models on multiple input resolutions significantly improves detection accuracy. This indicates that multi-scale approaches are necessary to handle variation in object size.

(Chuanxiang, et al., 2023) techniques such as image tiling are commonly used in aerial detection tasks. By dividing large images into smaller patches, the relative size of the marker increases, making it easier for the model to detect.

(Lin, 2017) Modern detection architecture also incorporates Feature Pyramid Networks (FPN), which allow models to detect objects at multiple scales by combining information from different feature layers.

False Positives in Complex Backgrounds

(Lin, 2017) False positives are a major issue in environments such as construction sites, where patterns like GCP markers may be present. Research in object detection shows that models trained on diverse datasets with challenging background examples perform better at distinguishing target objects from noise.

(Redmon, 2016) Post-processing methods also play a role in improving detection reliability. Techniques such as Non-Maximum Suppression (NMS) are used to remove overlapping detections, while confidence thresholding helps filter out low-probability predictions. These methods reduce incorrect detections without significantly impacting recall.

Evaluation and Validation

Evaluation of a GCP detection system must address two distinct aspects: classification performance and localisation accuracy. For classification, the standard metrics True Positive Rate (TPR), Recall, True Negative Rate (TNR), and False Positive Rate (FPR) each measure a different facet of model behaviour. (Lin, 2017) and (Redmon, 2016) both demonstrate that these metrics must be evaluated at a calibrated confidence threshold, because the precision-recall trade-off shifts with threshold selection and a single fixed value may not reflect operational performance.

For localisation, standard bounding box metrics such as Intersection over Union (IoU) are insufficient on their own. (Odriozola et al., 2024) evaluate their model specifically on the accuracy of predicted GCP centre coordinates, comparing model output against ground truth labels placed by experienced photogrammetry operators. This distinction is important: a detection can be scored as a true positive under a generous IoU threshold while still placing the predicted centre several pixels from the true centre an error that would directly degrade georeferencing accuracy. Validation must therefore include a separate localisation error metric, such as mean Euclidean distance between predicted and ground truth centre points, evaluated on a held-out test set drawn from operational conditions.

System Integration and Scalability

A GCP detection system does not operate in isolation it is a component within a larger photogrammetric processing pipeline that includes flight planning, image acquisition, structure-from-motion reconstruction, and orthorectification. (Odriozola et al., 2024) describe exactly this integration challenge: their automated detection pipeline must process a large collection of drone-based images and feed detected GCP centres into the subsequent georeferencing step, making throughput and

For the system developed in this project, scalability considerations include how the tiling preprocessing step (Chuanxiang et al., 2023) interacts with total processing time when handling a full set of flight images, and what model complexity is realistic given the available hardware.

Available product analysis

The products that will be analysed here fall into 3 categories model design, model evaluation and model integration. These were chosen because data collection and preparation will be discussed further along in this research, and marker design was researched separately already.

model design

for model designs used for object detection (and identification) there are a few big options. (Zhao, Yan Zhou, & Chen, 2017)

- Pretrained single pass
- From scratch single pass
- Two staged pretrained
- Two staged from scratch

Generally speaking, two staged models have a higher accuracy but take longer at inference. While single pass models can more easily do real time detection and classification.

Pretrained models

Deep learning-based object detection models are the standard approach for automated GCP detection.

(Chuanxiang et al., 2023) used a YOLOv5-based model for GCP detection and demonstrated high accuracy across aerial images. One-stage detectors such as YOLO are effective because they perform detection in a single pass, allowing efficient processing of large UAV datasets.

(Odriozola et al., 2024) used a ResNet-152-based model and achieved highly precise localization results, indicating that deep convolutional architectures can effectively capture features required for accurate GCP marker detection.

Untrained models

The architecture for small CNN's varies a lot, but often they use at least 3 convolutional layers with pooling layers in between these are then followed by dense connected layers.

When it comes to large models these have big collections of multiple convolutional and pooling layers, and a lot of large dense connected layers (ZHAO, ZHOU, & CHEN, 2017). For example, VGG16 has 13 convolutional layers with 5 max pooling layers.

The larger models benefit more from transfer learning making it less interesting to train these from scratch for each use case. Though small candidate generating models can be trained on use case specific data and reach comparable performance to similarly sized transfer learning then fine-tuned models.

System Integration and Deployment

In addition to detection performance, practical implementation requires that models can be integrated into existing software systems. There are a few file formats that can be used to export or import neural networks, such as: ONNX, keras, and h5 (or HDF5). These allow the same model to be deployed across different platforms without modification.

Architecture sketching

The first step in the pipeline would be to match the input to the prepared data. This means tiling it like mentioned in the literature study. Then a multi pass approach for detecting and identifying the GCP will be used. Because as stated in the available products analysis, those kinds of systems get higher overall accuracy. So a from scratch filtering model to generate candidates will be used. Then a marking model will find the GCP markers, to the let an identification model figure out which GCP marker it is.



Figure 1 Overview of proposed detection pipeline

This filter model should greatly reduce cost by preventing empty images from reaching the 2 much bigger models for marking and identifying them.

Conclusion

In conclusion, the solution is a four-step pipeline:

- **Tiling:** Large images are cut into smaller segments. This makes the markers look larger and easier to find.
- **Filtering:** A tiny, lightweight custom model quickly scans the blocks and throws away the empty ones. This saves a lot of processing time and stops false positives early.
- **Detection:** Only the blocks containing a marker are sent to the fine-tuned YOLO model for GCP marker training and testing.
- **Identification:** Taking the detected GCP's and identifying which of the known GCP's it is.

How does the data need to be prepared for GCP marker?

The goal of this section is to find out how the data from the MRR test site should be prepared for best performance of the models. Here there is also a focus on making sure it's a reproducible way of preparing the data, that doesn't needlessly bias the models.

Decomposition:

The preparation of UAV image data for GCP marker detection can be divided into the following key steps and factors:

- Image selection and filtering
- Metadata extraction and processing
- Image resizing and normalization
- Handling scale variation
- Image tiling (patch generation)
- Data augmentation
- Dataset diversity
- Annotation of GCP markers
- Annotation format conversion
- Data quality validation

Literature study

Before images can be used to train a YOLO-based GCP marker detection model, several preparation steps are required to transform raw UAV imagery into a format the model can process.

Image selection and filtering

Not all images captured during a UAV flight are suitable for training or detection. (Sieberth et al., 2016) identify blur caused by camera movement from wind, turbulence, or sudden flight inputs as one of the main problems preventing full automation of UAV image processing and show that blurred images degrade the accuracy of photogrammetric algorithms. Removing such images before training is therefore a necessary first step. Beyond blur, images that are overexposed or only partially cover the survey area should also be excluded, as they introduce misleading or incomplete data into the training set.

Metadata extraction and processing

This literature study looks at what kind of different data we need when flying drones for mapping. Every raw drone photo automatically saves important flight information inside hidden text layers in the file which is called **Metadata**. This information includes GPS coordinates, flying height, and camera angles. Separating this flight data from the actual picture lets us see exactly how flight details change how the ground markers look before we start working with the images.

altitude and scale variance

To analyse exactly how scale changes affect performance, (Chuanxiang, et al., 2023) filtered their data into eight 20-meter altitude steps between 60 and 220 meters. Their results showed high accuracy at most heights, keeping a precision above 97.4% and a recall above 91%. However, a major drop in recall down to 72% occurred specifically at the 140–160 meter altitude range. This error happened because the model was trained only on flat landscapes (like roads and settlements) but was tested on steep, sloping hillsides at that specific height.

This proves that altitude metadata must be balanced with terrain slope data to keep detection reliable. Because of this, including images of different landscapes in the training dataset is necessary to improve the robustness of the model. In addition, their tests showed that across the entire 60 to 220-meter flying range, the smallest target marker size the model can successfully detect must be greater than 12×12 pixels on the screen.

For commercial DJI Mavic and Mini platforms, this calculation can optionally be bypassed by extracting the proprietary RelativeAltitude tag embedded inside the custom DJI XMP metadata layer.

camera angles and perspective distortion

The orientation and viewing angle of the drone's camera heavily dictate how ground shapes are geometrically depicted. (Gromada, 2022) find that GCP markers frequently appear rotated or geometrically skewed across the frame depending directly on the camera angle.

Because standard horizontal bounding boxes fail to tightly capture the skewed outlines of these deformed shapes, literature recommends adopting oriented bounding boxes (OBB) to structurally adapt to the camera's tilt parameters.

environmental noise and data diversity

Determining data diversity requirements requires explicitly representing environmental and operational variations within the dataset. (Odriozola, Pauly, Oswald, & Raymaekers, 2024) identify drone flight altitudes, diverse lighting setups, and potential physical obstructions on the ground markers as pivotal dataset variables that must be represented to ensure real-world pipeline adaptability.

Furthermore, (Sieberth, 2016) demonstrate that quality checks must identify flight-induced image degradation like motion blur. Motion blur caused by camera movement from wind, turbulence, or sudden flight inputs directly degrades the accuracy of automated processing algorithms, making it necessary to filter out these corrupted frames before data ingestion.

Available Product Analysis

The products that will be analysed here fall into 3 categories: standard header parsers, vendor-specific telemetry engines, and geospatial mapping libraries. These were chosen because programmatic metadata extraction requires specific code frameworks to separate embedded string blocks from the underlying image pixel matrix.

standard header parsers

For reading basic image container blocks, there are a few lightweight open-source options:

- ExifRead
- Piexif

Generally speaking, these native Python script libraries feature ultra-low computational overhead. They allow developers to quickly parse standard EXIF headers to extract global coordinates, timestamps, and base camera model specifications from raw image files.

vendor-specific telemetry engines

(PyExifTool Wrapper) Advanced metadata isolation relies on comprehensive command-line parsing utilities like ExifTool interfaced via a Python script wrapper. This engine can read embedded XMP (Extensible Metadata Platform) strings where specific drone manufacturers like DJI write custom flight telemetry. This tool allows developers to extract real-time gimbal pitch, roll, yaw, and relative flight heights directly from the raw asset directory.

geospatial mapping libraries

The Geospatial Data Abstraction Library (GDAL) provides a robust programming framework to process complex imagery files. GDAL operates as a backend translation engine to programmatically read attached metadata scans, extracting precise pixel resolution values and spatial coordinate reference frameworks directly from complex remote sensing layers.

Comparative Analysis

Evaluating metadata extraction architectures and target metrics reveals clear trade-offs in capability, data structure efficiency, and pipeline accuracy.

extraction capability and library overhead

- **ExifRead and Piexif:** Provide highly efficient, fast execution but are completely blind to custom vendor XMP layers, making them incapable of extracting camera gimbal angles or pitch metrics.
- **ExifTool:** Highly effective at extracting deep text-based camera fields and hidden drone parameters. However, it relies on a separate background program (exiftool.exe), requiring you to manually install an external application on your system path for your Python code to call.
- **GDAL:** (Gromada, 2022) is a standard framework for verifying exact pixel resolutions and coordinate tracking from attached image metadata. Unlike ExifTool, it runs directly inside Python and does not require an external program running alongside it.

Image resizing and normalisation

Before images are passed to the model, pixel values are standardized to ensure consistent input. (Chuanxiang et al., 2023), following the YOLT method, pad and crop original images of 5280*3956 pixels into smaller patches to make them processable. Image pixel values are then normalized to a consistent range, a standard step in deep learning pipelines that stabilizes model training.

Handling scale variation

GCP markers change in apparent size depending on flight altitude. (Chuanxiang et al., 2023) collected UAV images of GCP markers from altitudes ranging from 60 to 220 meters to ensure the model was exposed to the full range of scale variation it would encounter in deployment.

Annotation format

Before training can begin, every object of interest in every training image must be manually labelled. (Mathies, 2024) describes this process for YOLO-based models: each object is identified by drawing a bounding box around it and assigning it a class label. The annotation is stored as a text file per image, where each line represents one object in the format: class index, centre x, centre y, width, and height all normalised to values between 0 and 1 relative to the image dimensions. This normalised format ensures that positional values remain valid if the image is rescaled.

(Mathies, 2024) also highlights several annotation rules that directly affect model quality:

- all instances of a class must be labelled in every image, as partial labelling confuses the model
- bounding boxes should fit as tightly as possible around the object
- and background images (images that contain no markers) should be included in the dataset at a proportion of 0 to 10% to help the model learn to suppress false positives.

For GCP markers this means drawing a bounding box around each marker and converting the pixel coordinates to this normalised format. (Chuanxiang et al., 2023) additionally used oriented bounding boxes (OBB) rather than standard horizontal boxes, because GCP markers in UAV imagery can appear rotated depending on camera angle standard boxes do not accurately capture the marker outline in these cases.

Annotation format conversion

(Chuanxiang et al., 2023) apply the YOLOv5-OBB format, which requires each image to have a corresponding label file where object locations are expressed as normalized coordinates. Where annotations are created in other formats using general annotation tools, they must be converted to the format required by the chosen model before training can begin.

Data augmentation

(Odriozola et al., 2024) show that augmenting training images by simulating different lighting conditions and marker orientations improves the model's ability to generalize across conditions that cannot be fully covered through data collection alone.

Available product analysis

To address the time-consuming nature of manual data annotation, we evaluated several software options to find the best fit for our pipeline. While multiple paid tools offer features like automation support and collaboration utilities, their subscription constraints and paywalled functionalities made them unsuitable for our workflow.

Instead, we selected the open-source platform **Label Studio**. It enables efficient data labelling and natively supports exporting directly into the YOLO format and other common dataset structures required for our model training.

Data preparation

We used label studio to annotate the data. After exporting, the folder contains the Image and label file associated with it which contains the exact coordinates of GCP markers in the image.

Commented [SG5]: Oriented bounding boxes and all? I though you only did vertical boundingboxes?

After annotating the data, we tested it on YOLOv8n model.

The results are:

- Precision (B): 87.24%
- Recall (B): 74.90%
- mAP50 (B): 81.82%
- mAP50-95 (B): 37.05%
- Fitness: 37.05%

Tiling

As seen above, the result with the big images (up to 5280*3956 in resolution) isn't the best and doesn't meet the final requirements. Also, the images we have as training data contain more GCP markers than most real-world applications.

Thus, the images were cut up into smaller tiles 512x512. This was done with some overlap to make sure any marker would show up well enough in at least one of the tiles. There are two main effects that this approach has. First is that any tile with a GCP marker only has one, as would be the case in the real-world application. Second the GCP markers in the tiles are proportionally bigger in the image. Thus, increasing the signal to noise ratio.

Afterwards we annotated the tiled Images, we fine-tuned the YOLO v8 model with it. These are the results:

- Precision (B): 97.66%
- Recall (B): 100.00%
- mAP50 (B): 99.50%
- mAP50-95 (B): 90.28%
- Fitness: 90.28%

This shows that Image tiling has improved the overall accuracy by a big margin.

Generation of synthetic data

For the smaller filter model, the time was taken to generate synthetic data because this wouldn't take extra hand annotating.

This synthetic data was generated by rotating each image to 3 different multiples of 90 degrees. This left us with 4 images for each of the original images leaving us with 300% more data. This is quite aggressive but is a common way of taking away possible rotational bias in the data. This method can be done for a larger dataset with a less aggressive original to synthetic data ratio.

Conclusion

the Geospatial Data Abstraction Library (GDAL) should be used because it can extract and process pixel resolutions, altitudes, and camera angles simultaneously within a single framework. This provides an easy method for handling UAV flight metadata to better understand which metadata parameters influence accuracy.

The tiling of the images works well to both make the filter model more manageable and make all models more accurate. Thus, the tiling is a must have for the final product.

The data should also have more variation. This it for the models to not be biased based by altitude, background, marker rotation, and lighting condition, a good evenly distributed variation of these factors should be included in the dataset.

How can the data best be used to detect future GCP marker?

After data is captured and prepared, the next step is determining how to best utilize it within an object detection framework to recognize future GCP markers. This requires selecting the right machine learning model architecture and training strategy. Because drone images are high-resolution, but markers appear very small, the data format directly impacts the model's success.

This section explores the optimal model selection process and references the empirical testing results used to find the most accurate and reliable configuration.

YOLO model

(Winklaar, 2026) To find the best approach, experiments were conducted across multiple generations of YOLO models using both raw (non-tiled) and pre-processed (tiled) data strategies.

The testing yielded the following key insights regarding model performance:

- **The Impact of Tiling:** Initial testing using standard, non-tiled imagery with YOLOv8n failed completely. Shrunk down aerial images cause small target markers to lose critical pixel details, making them undetectable. Moving to a tiled data strategy resulted in massive accuracy gains across all models.
- **Model Comparison:** Older iterations of the YOLO framework heavily outperformed both the newer YOLO11 and the latest YOLO26 architectures. Specifically, YOLO11x suffered from high false-positive rates by confusing natural terrain features with markers, while YOLO26m failed to achieve high accuracy.

Models based on the YOLOv5 architecture proved to be the most robust and reliable. Both **YOLOv5n** and **YOLOv5x** achieved stable convergence during validation and provided highly reliable, precise predictions on the test sets.

Filter model

For the filter model one of the priorities was to make it as small as possible with a reasonable accuracy. This is because that would save the most computational power over sending all of the images to the bigger models. Thus the first step was to select an architecture. In the results in (figure 2) the results of 9 different models are listed.

The 1cov model only has 1 convolutional layer and pooling layer followed by a dense layer of 128 neurons. The rest of the names follow a pattern where the number is the size of the dense layers and a “d” afterward shorthand for double, tells you if they have 1 or 2 dense layers. These later models all have the same first 6 layers, which are 3 convolutional layers with pooling layers in between.

Evaluate

```
: model1conv.evaluate(test_ds)
95/95 ————— 5s 48ms/step - accuracy: 0.7916 - loss: 0.7484
: [0.7250763177871704, 0.7931578755378723]

: model128.evaluate(test_ds)
95/95 ————— 6s 60ms/step - accuracy: 0.9629 - loss: 0.1933
: [0.18614423274993896, 0.9663158059120178]

: model128d.evaluate(test_ds)
95/95 ————— 6s 60ms/step - accuracy: 0.9519 - loss: 0.4825
: [0.4895533621311188, 0.9521052837371826]

: model164.evaluate(test_ds)
95/95 ————— 6s 59ms/step - accuracy: 0.9603 - loss: 0.2056
: [0.1786353439092636, 0.9631578922271729]

: model164d.evaluate(test_ds)
95/95 ————— 6s 59ms/step - accuracy: 0.9540 - loss: 0.3010
: [0.24813444912433624, 0.9605262875556946]

: model132.evaluate(test_ds)
95/95 ————— 6s 58ms/step - accuracy: 0.9571 - loss: 0.2142
: [0.2284180223941803, 0.9568421244621277]

: model132d.evaluate(test_ds)
95/95 ————— 6s 58ms/step - accuracy: 0.9671 - loss: 0.2956
: [0.22912189364433289, 0.9700008286102295]

: model116.evaluate(test_ds)
95/95 ————— 6s 59ms/step - accuracy: 0.5348 - loss: 0.6909
: [0.689574122428894, 0.5473684072494507]

: model116d.evaluate(test_ds)
95/95 ————— 6s 58ms/step - accuracy: 0.9548 - loss: 0.2550
: [0.2008652240037918, 0.9573684334754944]
```

Figure 2 model architecture performance comparison

The model with 2 dense layers of 16 neurons was chosen because it has very little parameters, yet has an accuracy that is comparable to the bigger models. This is probably because the bigger models have a subnetwork of similar size to the double 16 models that does all the work.

After architecture selection we did hyperparameter tuning. Because of time and computational power restraints, the hyper parameters where tuned separately. Ideally, this would be done in one big grid search. The hyperparameters that where tuned are:

- Pooling layer activation
- Dense layer activation
- Output layer activation
- Optimizer
- Epochs

First the pooling and dense layers where tuned in a grid search together with the following results:

```

pooling layer activation: relu - deep layer activation: relu
95/95 ————— 6s 61ms/step - accuracy: 0.9666 - loss: 0.1768
pooling layer activation: relu - deep layer activation: sigmoid
95/95 ————— 6s 59ms/step - accuracy: 0.9686 - loss: 0.1006
pooling layer activation: relu - deep layer activation: tanh
95/95 ————— 6s 58ms/step - accuracy: 0.9674 - loss: 0.1459
pooling layer activation: relu - deep layer activation: elu
95/95 ————— 6s 58ms/step - accuracy: 0.9597 - loss: 0.1987
pooling layer activation: relu - deep layer activation: softplus
95/95 ————— 6s 58ms/step - accuracy: 0.9575 - loss: 0.1930
pooling layer activation: sigmoid - deep layer activation: relu
95/95 ————— 6s 59ms/step - accuracy: 0.5523 - loss: 0.6893
pooling layer activation: sigmoid - deep layer activation: sigmoid
95/95 ————— 6s 60ms/step - accuracy: 0.4678 - loss: 0.6938
pooling layer activation: sigmoid - deep layer activation: tanh
95/95 ————— 6s 60ms/step - accuracy: 0.4520 - loss: 0.6941
pooling layer activation: sigmoid - deep layer activation: elu
95/95 ————— 6s 60ms/step - accuracy: 0.5279 - loss: 0.6917
pooling layer activation: sigmoid - deep layer activation: softplus
95/95 ————— 6s 59ms/step - accuracy: 0.5669 - loss: 0.7526
pooling layer activation: tanh - deep layer activation: relu
95/95 ————— 6s 59ms/step - accuracy: 0.7713 - loss: 1.0324
pooling layer activation: tanh - deep layer activation: sigmoid
95/95 ————— 6s 59ms/step - accuracy: 0.5411 - loss: 0.6927
pooling layer activation: tanh - deep layer activation: tanh
95/95 ————— 6s 59ms/step - accuracy: 0.4605 - loss: 0.6964
pooling layer activation: tanh - deep layer activation: elu
95/95 ————— 6s 59ms/step - accuracy: 0.7766 - loss: 1.0606
pooling layer activation: tanh - deep layer activation: softplus
95/95 ————— 6s 59ms/step - accuracy: 0.7874 - loss: 0.9263
pooling layer activation: elu - deep layer activation: relu
95/95 ————— 6s 61ms/step - accuracy: 0.7408 - loss: 1.0214
pooling layer activation: elu - deep layer activation: sigmoid
95/95 ————— 6s 61ms/step - accuracy: 0.4648 - loss: 0.6943
pooling layer activation: elu - deep layer activation: tanh
95/95 ————— 6s 61ms/step - accuracy: 0.5360 - loss: 0.6922
pooling layer activation: elu - deep layer activation: elu
95/95 ————— 6s 61ms/step - accuracy: 0.7698 - loss: 1.2542
pooling layer activation: elu - deep layer activation: softplus
95/95 ————— 6s 61ms/step - accuracy: 0.7965 - loss: 1.2687
pooling layer activation: softplus - deep layer activation: relu
95/95 ————— 7s 70ms/step - accuracy: 0.5538 - loss: 0.6892
pooling layer activation: softplus - deep layer activation: sigmoid
95/95 ————— 7s 70ms/step - accuracy: 0.5340 - loss: 0.6924
pooling layer activation: softplus - deep layer activation: tanh
95/95 ————— 7s 70ms/step - accuracy: 0.4626 - loss: 0.6933
pooling layer activation: softplus - deep layer activation: elu
95/95 ————— 7s 70ms/step - accuracy: 0.5368 - loss: 0.6912
pooling layer activation: softplus - deep layer activation: softplus
95/95 ————— 7s 70ms/step - accuracy: 0.8060 - loss: 0.4603

```

Figure 3 pooling and deep layer activation results

From these the combination ReLu pooling layer activation with than dense activation was chosen as it has the highest accuracy. With these chosen activations, we then tuned the optimizer with the following results:

```

optimizer: adam
95/95 ————— 6s 60ms/step - accuracy: 0.9747 - loss: 0.1246
optimizer: sgd
95/95 ————— 6s 59ms/step - accuracy: 0.8248 - loss: 0.4119
optimizer: ftrl
95/95 ————— 6s 59ms/step - accuracy: 0.5501 - loss: 0.6911
optimizer: lion
95/95 ————— 6s 59ms/step - accuracy: 0.5534 - loss: 0.6918
optimizer: nadam
95/95 ————— 6s 59ms/step - accuracy: 0.9703 - loss: 0.1222

```

Figure 4 optimizer results

Here adam seems to have been the best although nadam was very close. Thus for further fine-tuning adam was used. But in final training both adam and nadam were used.

After optimizer, output activation was fine-tuned with following results:

```
out_act: softmax
95/95 ————— 6s 62ms/step - accuracy: 0.9578 - loss: 0.1591
out_act: relu
95/95 ————— 6s 59ms/step - accuracy: 0.5448 - loss: 0.6931
out_act: softplus
95/95 ————— 6s 58ms/step - accuracy: 0.9701 - loss: 0.1122
```

Figure 5 output layer results

Clearly softplus has the best accuracy. But in training test and eval accuracy where quite far apart (.9932 vs .9558) thus softmax was chosen as it's accuracy during training was a bit closer together (.9885 vs .9568).

Then the number of epochs where tuned with the following results:

```
epochs: 0
95/95 ————— 6s 60ms/step - accuracy: 0.9685 - loss: 0.1910
epochs: 5
95/95 ————— 6s 60ms/step - accuracy: 0.9686 - loss: 0.1856
epochs: 10
95/95 ————— 6s 59ms/step - accuracy: 0.9685 - loss: 0.1847
epochs: 15
95/95 ————— 6s 58ms/step - accuracy: 0.9658 - loss: 0.1881
epochs: 20
95/95 ————— 6s 58ms/step - accuracy: 0.9653 - loss: 0.1994
epochs: 25
95/95 ————— 6s 59ms/step - accuracy: 0.9704 - loss: 0.1624
epochs: 30
95/95 ————— 6s 59ms/step - accuracy: 0.9677 - loss: 0.1851
epochs: 35
95/95 ————— 6s 59ms/step - accuracy: 0.9653 - loss: 0.1856
epochs: 40
95/95 ————— 6s 59ms/step - accuracy: 0.9656 - loss: 0.2139
epochs: 45
95/95 ————— 6s 58ms/step - accuracy: 0.9688 - loss: 0.1780
epochs: 50
95/95 ————— 6s 59ms/step - accuracy: 0.9655 - loss: 0.1784
epochs: 55
95/95 ————— 6s 59ms/step - accuracy: 0.9689 - loss: 0.1758
epochs: 60
95/95 ————— 6s 59ms/step - accuracy: 0.9663 - loss: 0.1896
epochs: 65
95/95 ————— 6s 59ms/step - accuracy: 0.9722 - loss: 0.1645
epochs: 70
95/95 ————— 6s 58ms/step - accuracy: 0.9661 - loss: 0.1839
epochs: 75
95/95 ————— 6s 59ms/step - accuracy: 0.9703 - loss: 0.1578
epochs: 80
95/95 ————— 6s 59ms/step - accuracy: 0.9676 - loss: 0.1927
epochs: 85
95/95 ————— 6s 59ms/step - accuracy: 0.9676 - loss: 0.2046
epochs: 90
95/95 ————— 6s 59ms/step - accuracy: 0.9695 - loss: 0.1687
epochs: 95
95/95 ————— 6s 58ms/step - accuracy: 0.9724 - loss: 0.1470
```

Figure 6 epoch results

In the evaluation the number is 5 lower then the epochs run because of how it was calculated. Thus the number of epochs before the accuracy took a drop is at 30 (25 in the image) so 30 epochs was chosen for further training.

Final training was then evaluated but on the test dataset, and a purposefully difficult test dataset from a different site with actual construction equipment on site including many square objects for the model to be confused by. These gave the following results:

```
: finalmodel.evaluate(test_ds)
  finalmodel_softplus.evaluate(test_ds)
```

95/95 ————— 6s 59ms/step - accuracy: 0.9625 - loss: 0.1584
95/95 ————— 6s 60ms/step - accuracy: 0.9573 - loss: 0.1795

Figure 7 final training results on test data

```
finalmodel.evaluate(fresh_ds)
finalmodel_softplus.evaluate(fresh_ds)
```

33/33 ————— 2s 60ms/step - accuracy: 0.8049 - loss: 0.6729
33/33 ————— 2s 57ms/step - accuracy: 0.7182 - loss: 0.8452

Figure 8 final training results on fresh data

The test accuracy is quite high. The fresh data from the difficult dataset shows that some transferrable learning was done. Although for proper functioning more data from real sites will have to be used for training.

Conclusion

Although both models have promising accuracy on the test set, they do suffer on the fresh set. Thus the learning from the current set is poorly transferable to the real world use case.

The current training methods with YOLOv5x seem to be the best for the object detection model. As for the filter model, the hyperparameters should be tuned properly in a training environment that can handle it.

Conclusion

So for automatic detection of GCP models in a robust way, the following pipeline should be built: tiling -> filtering -> detection -> identification.

The method of tiling that has been devised can deal with varied sizes of images thus, in theory, working for any possible drone used to survey an area.

As for filtering a small model can be created to filter out images without GCP markers, although the right data needs to be collected for the accuracy to be high enough.

The detection model that seems to be the best is YOLOv5x, thus this should be used. But again to get the accuracy high enough different data has to be collected.

Future Steps

Some key problems have been found in this research, namely:

- **Dataset size:** there isn't enough data to ensure stable training and high accuracy.
- **Data quality:** the data that was used was not representative of the real world use case. This led to poorly transferable learning.
- **Hardware/time for training:** the filter model couldn't be properly tuned due to a combination of hardware limitations and a lack of time. A proper grid search on a full set of hyperparameters should be performed.

On top of these issues there wasn't enough time to start the building of the identification model.

With that, the following activities have to be done to get to a proper functioning end product.

Data collection: MRR drones should start collecting annotated images of GCP markers through their hand marking that they are already doing. In this they should keep in mind that the companies they work with have to agree to it for this data to be used as training data.

Hyper parameter tuning: a proper grid search should be set up for the hyperparameter of the smaller model, including the parameters of the optimizers. This should if possible be done on a GPU.

Building identification: a system has to be built to go from a detection to a identification of a GCP marker.

Bibliography

- Aerotas. (n.d.). *Ground Control Points (GCPs)*. Retrieved from Aerotas:
<https://www.aerotas.com/store/aerotas-ground-control-points>
- amazon. (n.d.). *10 Pack Drone Ground Control Point GCP, 24"x24" Black/White Markers UAV Aerial Target for Mapping & Surveying*. Retrieved from amazon:
https://www.amazon.com/Cyiwniao-WhiteMarkers-Surveying-Compatible-DroneDeploy/dp/B0DHXXYGSD/ref=sr_1_3?sr=8-3&th=1
- Chuanxiang, C., Jia, Y., Chao, W., Zhi, Z., Xiaopeng, L., Di, D., . . . Zhiheng, Z. (2023). Automatic detection of aerial survey ground control points based on Yolov5-OBB.
- Lin, T.-Y. (2017). *Feature Pyramid Networks for Object Detection*. Retrieved from Arxiv:
<https://arxiv.org/pdf/1612.03144>
- Odriozola, G. M., Pauly, K., Oswald, S., & Raymaekers, D. (2024). Automating Ground Control Point Detection in Drone Imagery: From Computer Vision to Deep Learning.
- Redmon, J. (2016, May 9). *You Only Look Once*. Retrieved from arxiv.org:
<https://arxiv.org/pdf/1506.02640>
- Sieberth. (2016, December). *Automatic detection of blurred images in UAV image sets*. Retrieved from Science Direct:
<https://www.sciencedirect.com/science/article/abs/pii/S0924271616303999>
- Sky high bulls eye. (n.d.). *UAV Harlequin Ground Control Points/Aerial Mapping & Surveying (24"x24") | Center Passthrough | Numbered 0-9 (10 Pack)*. Retrieved from Sky high bulls eye: <https://skyhighbullseye.com/products/aerial-mapping-24x24-harlequin-numbered-gcps-center-passthrough>
- Tushev, S., Sukhovilov, B., & Sartasov, E. (2019). Robust coded target recognition in adverse light conditions.
- Wang, Q., Liu, Y., Guo, Y., Wang, S., Zhang, Z., Cui, X., & Zhang, H. (2022). A Robust and Effective Identification Method for Point-Distributed Coded Targets in Digital Close-Range Photogrammetry.
- Zhao, K., Yan Zhou, & Chen, X. (2017). Object detection: training from scratch.

Figures

Figure 1 Overview of proposed detection pipeline.....	14
Figure 2 model architecture performance comparison.....	24
Figure 3 pooling and deep layer activation results.....	25
Figure 4 optimizer results	25
Figure 5 output layer results.....	26
Figure 6 epoch results	26
Figure 7 final trainig results on test data	27
Figure 8 final training results on fresh data	27